

Transaction

1. Feature Overview

Immutable ledger of credits/debits (and later transfers). **Not implemented** — empty package `com.bankone.transaction`. Deposits today only mutate `account` balances.

Status: Stub / Planned

2. Business Purpose

Provide audit-grade money movement history, feed reports/dashboard, enable withdraw/transfer.

3. User Workflow

Planned: Account detail → transaction list; deposit/withdraw create rows; sidebar Transactions route (nav stub today).

4. Execution Flow

Planned (target):

```
AccountController.deposit()
    ↓
AccountServiceImpl.deposit()
    ↓
TransactionService.createCredit()
    ↓
TransactionRepository.save()
    ↓
AccountRepository.save()
```

5. Database Tables

None yet. Planned: `transaction` / `ledger_entry` with FK to `account`.

6. REST APIs

None. Planned examples: `GET /accounts/{id}/transactions`, `POST /accounts/{id}/withdraw`.

7. Controllers

None (`com.bankone.transaction` empty)

8. Services

None

9. Repositories

None

10. DTOs

None

11. Entities

None

12. Utility Classes

None

13. Configuration Classes

None; will need `SecurityConfig` matchers

14. Validation Rules

Planned: amount > 0; sufficient balance for debit; status ACTIVE; currency match

15. Security Rules

Planned: align with accounts — write ADMIN/EMPLOYEE; read include MANAGER

16. Exception Handling

Reuse `GlobalExceptionHandler`

17. Logging

Planned structured log per posting

18. Audit Events

Ledger row **is** the financial audit; optional link to `audit` module for who/when metadata

19. Testing Strategy

When built: deposit creates +1 CREDIT; withdraw insufficient funds; concurrent balance safety

20. Future Extension Guide

First slice: CREDIT on `deposit` + opening credit on `openAccount` . Then withdraw. Then transfer + beneficiary.

Future Modification Guide

Requirement: Introduce first ledger on deposit

Item	Detail
Files	New under <code>com/bankone/transaction/**</code> ; modify <code>AccountServiceImpl.java</code>
Classes	<code>Transaction</code> , <code>TransactionService</code> , <code>TransactionRepository</code> , <code>AccountServiceImpl</code>
Methods	<code>deposit()</code> — after balance update call <code>transactionService.recordCredit(...)</code>
Impact	<code>DashboardServiceImpl</code> <code>todayTransactionCount</code> ; Accounts UI; docs API/schema

Call hierarchy (target)

```
AccountController.deposit()
    ↓
AccountServiceImpl.deposit()
    ↓
TransactionService.createTransaction()
    ↓
TransactionRepository.save()
    ↓
AccountRepository.save()
```